

中国科学技术大学
专业硕士工程实践论文

基于深度学习的文本作业自动评分系统

小组成员：张浩恩 张明哲 张享钦 赵柏杨

指导教师：翟建芳

完成时间：二〇二六年六月

University of Science and Technology of China
A dissertation for engineering practice

**a Deep Learning-Based Automated Text Assignment
Scoring System**

Team members: Haoen Zhang, Mingzhe Zhang, Xiangqin Zhang, Baiyang Zhao
Supervisor: Jianfang Zhai
Completion date: June, 2026

摘要

随着在线教育、课程作业平台和大规模考试场景的发展，文本类作业的评价需求持续增长。作文、简答题和论述题等开放式文本能够较好地考查学生的语言表达、逻辑组织和知识理解能力，但其评分过程高度依赖教师经验，存在人工阅卷工作量大、评分尺度难以完全一致、反馈生成滞后等问题。自动作文评分（Automated Essay Scoring, AES）试图利用自然语言处理和机器学习技术学习文本与人工分数之间的映射关系，在保证评分一致性的同时提升评价效率。近年来，预训练语言模型在语义表示学习方面表现突出，为构建面向文本作业的自动评分系统提供了新的技术路径。

本文围绕“基于深度学习的文本作业自动评分系统”展开设计与实现。系统以 ASAP 自动作文评分数据集为实验数据基础，采用 RoBERTa 预训练语言模型提取作文深层语义向量，并通过 Mean Pooling 将序列表示压缩为固定维度文本表示，再使用多层感知器（MLP）完成分数回归预测。训练阶段采用按题目组独立标准化分数、5 折交叉验证和 Quadratic Weighted Kappa（QWK）一致性指标进行评估；推理阶段支持已知题目组评分和未知题目归一化融合评分。系统实现方面，本文采用 Flask-RESTful 构建后端接口服务，使用 PyTorch 和 Transformers 封装模型推理流程，采用 SQLite 持久化保存评分提交与明细记录，并使用 Streamlit 构建可交互 Web 前端。系统提供单篇评分、批量 CSV 评分、文本统计分析、历史记录查询筛选、详情查看、CSV 导出和删除等功能。

实验与测试结果表明，RoBERTa+MLP 模型在 ASAP 数据集上能够取得约 0.75 的平均 QWK，达到与人工评分员一致性相近的水平；系统功能测试覆盖单篇评分、批量评分、历史管理、文本分析和数据导出等核心流程，能够满足文本作业自动评分原型系统的使用需求。本文工作验证了预训练语言模型在文本作业自动评分任务中的有效性，也为后续引入可解释反馈、多维评分和多语言作业评分提供了工程基础。

关键词：自动作文评分；深度学习；RoBERTa；Flask；Streamlit；Quadratic Weighted Kappa

ABSTRACT

With the rapid growth of online education, learning platforms and large-scale assessment scenarios, the demand for evaluating open-ended text assignments has increased significantly. Essays, short answers and argumentative responses can reflect students' language expression, logical organization and knowledge understanding, but manual grading is time-consuming and may suffer from inconsistency among graders. Automated Essay Scoring (AES) uses natural language processing and machine learning techniques to learn the mapping between text and human scores, thereby improving grading efficiency while maintaining scoring consistency.

This thesis designs and implements a deep learning-based automated text assignment scoring system. Based on the ASAP dataset, the system uses RoBERTa to extract deep semantic representations of essays, applies mean pooling to obtain fixed-length vectors, and adopts a multilayer perceptron (MLP) for score regression. During training, scores are standardized separately for each prompt, and five-fold cross validation is used with Quadratic Weighted Kappa (QWK) as the main evaluation metric. For system implementation, Flask-RESTful is used to build the backend API, PyTorch and Transformers are used to encapsulate model inference, SQLite is used for persistent storage, and Streamlit is used to implement the web interface. The system supports single-essay scoring, batch CSV scoring, text statistics, history query, detail review, CSV export and deletion.

Experimental and system testing results show that the RoBERTa+MLP model achieves an average QWK of about 0.75 on the ASAP dataset, which is close to the agreement level of human raters. The implemented system covers the main workflow of text assignment scoring and provides a practical prototype for further research on explainable feedback, multi-dimensional scoring and multilingual assessment.

KEY WORDS: Automated Essay Scoring; Deep Learning; RoBERTa; Flask; Streamlit; Quadratic Weighted Kappa

目 录

第 1 章 绪论.....	1
1.1 研究背景.....	1
1.2 国内外研究现状.....	1
1.3 论文研究内容与主要工作.....	2
1.4 论文组织结构.....	3
第 2 章 系统需求分析.....	4
2.1 系统概述.....	4
2.2 角色与业务流程分析.....	4
2.3 功能性需求分析.....	5
2.4 非功能性需求分析.....	7
第 3 章 系统概要设计.....	9
3.1 总体架构设计.....	9
3.2 模型训练与推理流程设计.....	9
3.3 功能模块设计.....	11
3.4 数据库设计.....	12
3.5 接口设计.....	14
第 4 章 系统详细设计与实现.....	15
4.1 开发环境与目录结构.....	15
4.2 系统类结构设计.....	15
4.3 单篇评分顺序流程设计.....	16
4.4 多篇评分顺序流程设计.....	17
4.5 历史记录顺序流程设计.....	18
4.6 模型服务详细实现.....	19
4.7 后端服务详细实现.....	20

4.8 前端页面详细实现.....	20
4.9 历史记录与导出实现.....	21
第5章 系统测试.....	22
5.1 测试环境与方法.....	22
5.2 模型评估测试.....	22
5.3 功能测试.....	24
5.4 非功能测试.....	25
第6章 总结与展望.....	26
6.1 总结.....	26
6.2 展望.....	26
参考文献.....	28
致谢.....	错误!未定义书签。

第 1 章 绪论

1.1 研究背景

在课程教学、在线学习和大规模考试中，文本作业是评价学生综合能力的重要载体。与选择题、填空题等客观题相比，作文、简答题和论述题能够体现学生对知识点的理解程度、材料组织能力、语言表达能力以及逻辑推理能力，因此被广泛应用于语文、英语、历史、政治、通识教育以及专业课程考核中。然而，文本作业的评分往往依赖教师或阅卷人员逐篇阅读与判断，评分周期长、人工成本高，特别是在班级规模较大、考试批改集中或线上作业提交频繁的场景下，传统人工评分方式难以满足及时反馈的需求。

人工评分还存在一致性问题。不同评分者对评分标准的理解可能存在差异，同一评分者在不同时间段也可能受到疲劳程度、上下文样本和主观偏好影响，导致评分波动。对于教学场景而言，学生不仅需要分数，还希望获得关于文本长度、词汇使用、表达丰富度等方面的反馈。若所有分析完全依赖教师人工完成，会进一步增加教学负担。因此，设计一个能够自动读取文本、预测分数、生成基础统计反馈并保存评分历史的系统，具有明确的工程实践价值。

自动作文评分技术的发展经历了从人工特征工程到深度学习方法的变化。早期 AES 系统通常依赖词汇、句法、篇章结构和拼写错误等人工设计特征，再利用线性回归、支持向量机等模型进行评分。这类方法可解释性较强，但特征构建成本高，泛化能力受领域和题目限制。深度学习方法能够自动学习文本表示，CNN、LSTM 等模型在一定程度上提升了语义建模能力，但对于长文本的全局上下文捕捉仍存在不足。近年来，以 BERT、RoBERTa 为代表的预训练语言模型通过 Transformer 自注意力机制学习上下文语义表示，在文本分类、回归和语义匹配任务中取得了较好效果，为自动评分系统提供了新的模型基础。

本文项目聚焦纯文本作业自动评分，选择 RoBERTa 作为语义特征提取器，结合轻量级 MLP 回归器完成分数预测，并将模型封装为可调用的 Web 服务。相比单纯的模型实验，工程实践更强调完整可用的系统闭环：用户能够在前端输入或上传文本，后端能够完成评分与文本分析，系统能够记录评分历史并支持查询、导出和删除。围绕这一目标，本文完成了从数据分析、模型训练、系统架构设计到前后端实现和测试验证的完整工程流程。

1.2 国内外研究现状

国外关于自动作文评分的研究起步较早，已经形成了较为丰富的系统和方法。早期代表性系统包括 e-rater、PEG 和 IntelliMetric 等，它们通常围绕作文质量构建语言学特征，并通过统计学习方法预测分数。此类系统在实际考试和写作辅助中具有一定应用基础，但其效果高度依赖特征工程、题目类型和语料质量，迁移到新的课程或评分标准时需要重新设计特征。

随着深度学习的发展，研究者开始使用 CNN、LSTM、层次化神经网络和注意力机制学习作文表示。与人工特征方法相比，深度学习模型能够从数据中自动学习语义模式，减少人工规则设计成本。Taghipour 和 Ng 将神经网络用于 AES 任务，Dong 等提出层次化文本模型以同时学习句子级和篇章级表示。此后，研究进一步扩展到多任务学习、评分维度建模、跨题目评分和反馈生成等方向。

预训练语言模型的出现进一步推动了 AES 技术发展。Transformer 结构通过自注意力机制建模文本中任意两个词之间的关系，能够更好地捕捉长距离依赖。BERT、RoBERTa 等模型在大规模语料上预训练，具备较强的通用语义表征能力。R2BERT 等研究表明，将预训练模型用于作文评分，并结合回归损失、排序损失等约束，可以提升评分一致性。近年来的综述研究也指出，深度学习 AES 在分数预测方面已取得显著进展，但可解释性反馈和跨领域泛化仍是重要挑战。

国内相关研究主要集中在智能教育、英语作文自动批改、简答题自动评分和在线教学评价等方向。部分系统已经能够提供语法纠错、词汇统计和句式建议等功能，但针对具体课程作业的轻量化、可扩展自动评分系统仍有实践空间。工程实践场景往往不只关注模型指标，还要求系统易部署、界面清晰、数据可追溯，并能够为教师后续分析提供历史记录和导出能力。因此，本文将预训练语言模型与 Web 系统集成，重点解决文本评分模型服务化、结果可视化和评分记录管理问题。

1.3 论文研究内容与主要工作

本文围绕基于深度学习的文本作业自动评分系统开展研究与实现，主要工作包括以下几个方面。

1. 完成数据集分析与评分任务建模。本文选用 ASAP 数据集作为实验基础，分析 8 个题目组的样本数量、分数范围、文本长度和评分分布，并根据不同题目组的评分区间差异设计独立标准化与反标准化策略。
2. 完成 RoBERTa+MLP 自动评分模型设计。系统使用 RoBERTa 提取文本深层语义表示，通过 Mean Pooling 得到 768 维作文向量，使用 $768 \rightarrow 256 \rightarrow 96 \rightarrow 1$ 的 MLP 回归器输出标准化分数，并使用 QWK 指标评估模型与人工评分的一致性。
3. 完成自动评分系统架构设计与实现。系统采用 Streamlit 前端、Flask-RESTful 后端、PyTorch/Transformers 模型推理层和 SQLite 存储层的分层架构，实现单篇评分、批量评分、文本分析、历史查询、详情查看、导出和删除等功能。
4. 完成系统测试与总结分析。本文从模型评估、接口功能、前端交互、数据持久化和非功能需求等角度进行测试，验证系统能够完成完整评分闭环，并分析系统局限与后续改进方向。

1.4 论文组织结构

本文共分为六章。第 1 章为绪论，介绍研究背景、国内外研究现状、主要工作和论文结构。第 2 章为系统需求分析，说明系统角色、业务流程、功能性需求和非功能性需求。第 3 章为系统概要设计，给出总体架构、模型流程、功能模块、数据库和接口设计。第 4 章为系统详细设计与实现，结合项目代码说明模型服务、后端接口、前端页面和历史记录管理的实现细节。第 5 章为系统测试，介绍测试环境、测试方法、模型评估结果、功能测试和非功能测试。第 6 章为总结与展望，总结本文工作并提出后续优化方向。

第 2 章 系统需求分析

2.1 系统概述

基于深度学习的文本作业自动评分系统面向教师、学生和系统管理者提供文本评分服务。系统的核心目标是将深度学习评分模型封装为可交互、可追溯、可扩展的 Web 应用，使用户能够在浏览器中完成文本提交、分数查看、文本统计分析和历史记录管理。系统处理对象主要为英文作文文本，同时在设计上保留对其他纯文本作业类型扩展的可能。

系统输入包括单篇作文文本或包含多篇作文的 CSV 文件。用户选择作文题目组后提交文本，系统调用后端模型服务完成评分，并返回预测分数、标准化分数、文本字符数、词数、句子数、段落数、平均句长、词汇丰富度和高频词统计等结果。对于题目未知的文本，系统通过遍历多个题目组的分数区间进行归一化融合，输出 0 - 100 范围内的综合分数。

系统输出不仅包括即时评分结果，还包括持久化的历史记录。每次评分提交都会在 SQLite 数据库中生成一条 submissions 记录，并在 records 表中保存对应的每篇作文明细。用户可以按照编号、题目组和提交类型查询历史记录，也可以查看评分详情、导出 CSV 或删除记录。通过历史记录模块，系统能够满足教师对批量作业评分结果留存和二次分析的需求。

2.2 角色与业务流程分析

系统参与者主要包括学生、教师和管理员三类。学生侧重于提交作文并查看评分结果；教师侧重于批量评分、查看统计结果、管理历史记录和导出数据；管理员侧重于维护模型、数据集、服务运行环境和系统配置。当前工程实现以教师和普通使用者的评分流程为主，管理员相关功能主要体现在后端服务部署、模型文件更新和数据库维护。

系统核心业务流程如下：用户进入 Streamlit 前端页面后，选择单篇评分或批量评分模式；前端将作文文本或 CSV 文件通过 HTTP 请求发送至 Flask 后端；后端完成参数校验和文件解析，调用 EssayScoringService 进行模型推理；模型服务使用 tokenizer 编码文本，RoBERTa 输出 hidden state，Mean Pooling 生成文本向量，MLP 输出标准化分数，再根据题目组进行反标准化和裁剪取整；后端同时完成文本统计分析，并将提交信息与评分明细写入 SQLite；最后前端展示分数、文本统计和高频词图表。

对于历史记录业务流程，用户进入历史记录页面后可按编号、题目组和提交类型筛选记录。前端调用历史列表接口获取分页数据，用户可进一步查看某次提交详情。若用户选择导出，前端根据详情接口返回的数据生成 UTF-8 with BOM 编码的 CSV 文件，保证 Excel 打开时中文列名和内容可正常显示。若用户选择删除，前端弹出确认对话框，确认后调用后端删除接口清理 submissions 与 records 中的相关数据。

2.3 功能性需求分析

系统功能性需求围绕“提交文本—模型评分—结果展示—历史管理”这一主流程展开。根据参与者与系统交互关系，可以将系统用例划分为评分类用例、结果查看类用例、历史记录类用例和维护类用例。学生主要使用单篇评分和查看结果功能；教师除单篇评分外，还需要批量评分、历史查询、详情查看和结果导出；管理员主要负责模型文件、数据集和系统运行环境维护。系统用例图如图 2.1 所示。

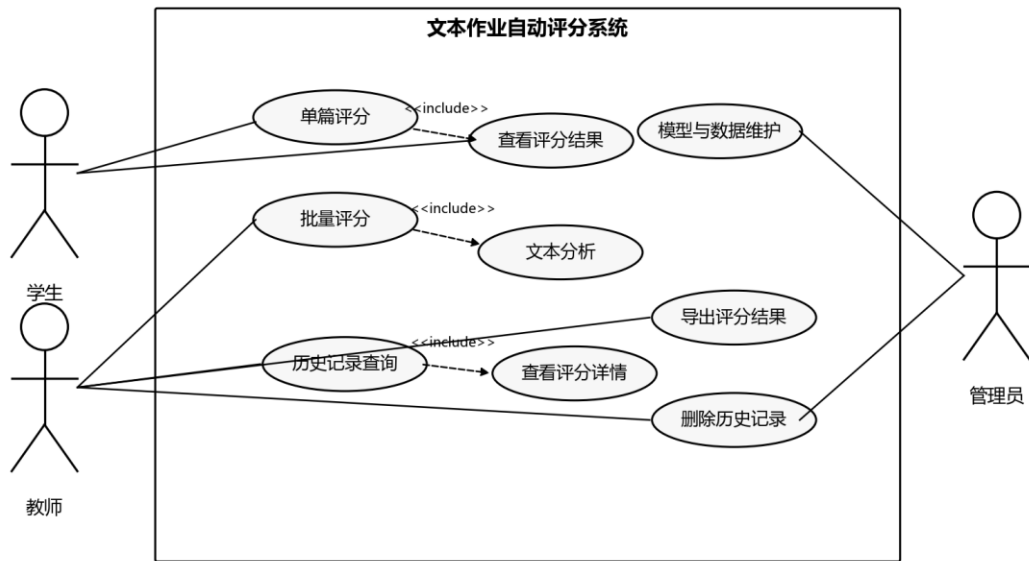


图 2.1 系统用例图

图 2.1 展示了文本作业自动评分系统的主要参与者和核心用例。单篇评分和批量评分都会包含评分结果查看与文本分析；历史记录查询进一步包含详情查看；导出和删除属于教师管理评分结果时常用的扩展操作。该用例图体现了系统从即时评分到历史管理的完整业务边界。

表 2.1 单篇评分用例规约

用例名称	单篇评分
参与者	学生、教师
前置条件	系统后端服务与前端页面正常运行，模型服务能够加载评分模型
后置条件	系统返回预测分数和文本分析结果，并在历史记录中保存一次单篇评分提交
基本流程	1. 用户进入单篇评分页面；2. 选择作文题目组或 unknown；3. 输入作文文本；4. 点击“开始评分”；5. 前端调用 /api/score/text；6. 后端调用模型服务完

	成评分；7. 系统展示分数、文本统计和高频词
可选流程	用户选择 unknown 时，系统对多个题目组的反标准化结果进行归一化融合，输出 0 - 100 分
异常流程	作文内容为空时，前端提示“请输入作文内容”；后端接收到空 essay 时返回 400 错误

表 2.2 批量评分用例规约

用例名称	批量评分
参与者	教师
前置条件	用户准备 CSV 文件，文件中每个非空单元格对应一篇待评分作文
后置条件	系统返回每篇作文的预测分数与文本分析结果，并归档上传文件
基本流程	1. 教师进入批量评分页面；2. 选择作文题目组；3. 上传 CSV 文件；4. 页面预览解析出的作文内容；5. 点击“开始评分”；6. 前端调用 /api/score/file；7. 后端解析 CSV 并逐篇评分；8. 系统展示批量评分结果
可选流程	教师可选择 unknown 模式处理题目组不确定的作文
异常流程	未上传文件时前端提示先上传 CSV；上传非 CSV 文件时后端返回“Only .csv is supported”；CSV 无有效文本时返回无作文内容错误

表 2.3 历史记录管理用例规约

用例名称	历史记录管理
参与者	教师、管理员
前置条件	系统数据库中已经存在评分提交记录
后置条件	用户完成历史记录查询、详情查看、导出或删除操作
基本流程	1. 用户进入历史记录页面；2. 输入编号或选择题目组、提交类型筛选条件；3. 点击查询；4. 系统分页展示历史提交；5. 用户可点击查看、导出或删除按钮执行进一步操作
可选流程	用户可不输入编号，仅通过题目组和提交类型筛选；也可直接按编号精确查询
异常流程	查询不存在的编号时返回记录不存在；删除不存在的记录时后端返回 404 错误

上述用例规约细化了系统最关键的业务场景。其中，单篇评分强调即时反馈，批量评分强调文件解析和多作文处理，历史记录管理强调评分结果的可追溯性。通过用例图和用例规约，可以明确系统边界、参与者职责以及异常处理要求，为后续概要设计和详细实现提供依据。

表 2.4 系统功能性需求表

功能需求	需求描述
单篇评分	用户选择作文题目组，输入一篇作文文本，系统返回预测分数和文本分析结果，并保存历史记录。
批量评分	用户上传 CSV 文件，系统逐篇解析文本并批量预测分数，返回每篇作文的编号、分数和分析结果。
题目选择	系统支持 1 - 8 个 ASAP 作文题目组，并支持 unknown 模式处理题目未知文本。
文本分析	系统统计字符数、词数、句子数、段落数、平均句长、词汇丰富度和 Top-10 高频词。
历史记录	系统支持分页查询评分历史，并可按编号、题目组和提交类型进行筛选。
详情查看	用户可查看某次提交下所有作文的原文、题目组、分数、标准化分数和文本分析。
结果导出	系统支持将历史评分明细导出为 CSV 文件，便于教师留存和二次分析。
记录删除	用户可删除不再需要的评分记录，系统同步删除对应评分明细。

从业务优先级看，单篇评分和批量评分是系统最核心的功能，它们直接决定系统是否能够完成文本作业自动评分任务。历史记录、详情查看和导出功能则面向教师实际使用过程中的管理需求，解决评分结果无法追溯和不便整理的问题。文本分析功能虽然不等同于深层写作反馈，但能够提供基础可解释信息，使用户了解系统评分之外的文本统计特征。

2.4 非功能性需求分析

表 2.5 系统非功能性需求表

非功能需求	需求描述
评分一致性	模型应在验证数据上取得较高 QWK，目标达到或接近人工评分员一致性水平。
响应时间	单篇评分应在可接受时间内返回结果，批量评分应能支持小规模 CSV 文件处理。
易用性	前端界面应清晰区分单篇评分、批量评分和历史记录，减少用户操作成本。
可维护性	模型推理、后端接口、前端页面和存储逻辑应分离，便于后续替换模型或扩展功能。

可靠性	系统应进行输入校验，对空文本、无文件、非 CSV 文件和非法参数返回明确错误信息。
可扩展性	后端采用 RESTful API，便于后续接入其他前端、课程平台或模型版本管理模块。
数据持久化	评分结果应保存至数据库，支持分页查询、筛选、详情查看和删除。
兼容性	导出文件应采用 UTF-8 with BOM 编码，保证常见办公软件可正确打开中文内容。

第 3 章 系统概要设计

3.1 总体架构设计

系统采用分层架构设计，自上而下分为前端展示层、后端服务层、模型推理层和数据存储层。前端展示层基于 Streamlit 实现，负责页面导航、输入控件、结果展示、统计图表和用户交互状态管理。后端服务层基于 Flask 和 Flask-RESTful 实现，负责 RESTful API 路由、请求参数校验、文件解析、文本分析、模型调用和响应组织。模型推理层基于 PyTorch 与 Hugging Face Transformers 实现，负责加载 RoBERTa、tokenizer 和 MLP 权重，并完成分数预测。数据存储层使用 SQLite 保存评分提交和作文明细，满足轻量级部署需求。

该架构具有实现简单、部署成本低、模块边界清晰等特点。Streamlit 适合快速构建数据应用和模型演示界面，Flask-RESTful 适合封装轻量级接口服务，SQLite 无需独立数据库服务即可完成持久化。对于工程实践项目而言，这种技术组合能够在较短周期内形成完整可运行系统，同时保留后续扩展为更复杂架构的空间。

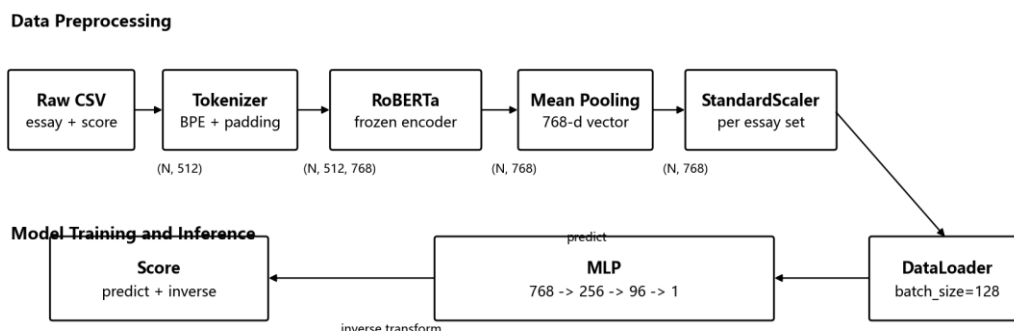


图 3.1 系统训练与推理数据流程图

3.2 模型训练与推理流程设计

本文模型流程包括数据读取、分数标准化、文本编码、向量池化、MLP 回归、反标准化和评分裁剪等步骤。训练数据来自 ASAP 数据集，包含 12,976 篇作文、8 个题目组以及人工评分。由于不同题目组的分数范围不同，例如 Set 1 的分数范围为 2 - 12，Set 8 的分数范围为 10 - 60，直接将原始分数作为统一回归目标会导致不同题目组之间尺度不一致。因此，训练阶段对每个题目组分别拟合 StandardScaler，将原始分数转化为均值为 0、方差为 1 的标准化分数。

文本编码阶段使用 roberta-base tokenizer 对作文进行 BPE 分词，并进行 padding 与 truncation。RoBERTa 输出每个 token 对应的 hidden state，形状为

$\text{batch_size} \times \text{seq_len} \times 768$ 。系统采用 Mean Pooling 对 token 维度取均值，得到每篇作文的 768 维向量表示。相比只使用 [CLS] 位置表示，Mean Pooling 能够利用所有 token 的上下文信息，对长短文本更加稳健。

回归器采用三层线性网络结构：Linear(768, 256)、ReLU、Dropout(0.3)、Linear(256, 96)、ReLU、Dropout(0.3)、Linear(96, 1)。训练损失函数为 MSELoss，优化器为 Adam，学习率为 $3e-4$ ，训练轮数为 20 epochs，并采用 5-fold 交叉验证评估模型稳定性。推理时，MLP 输出标准化分数，系统根据题目组对应的均值与标准差进行反标准化，并将分数裁剪到该题目组合法范围内，最后四舍五入为整数分。

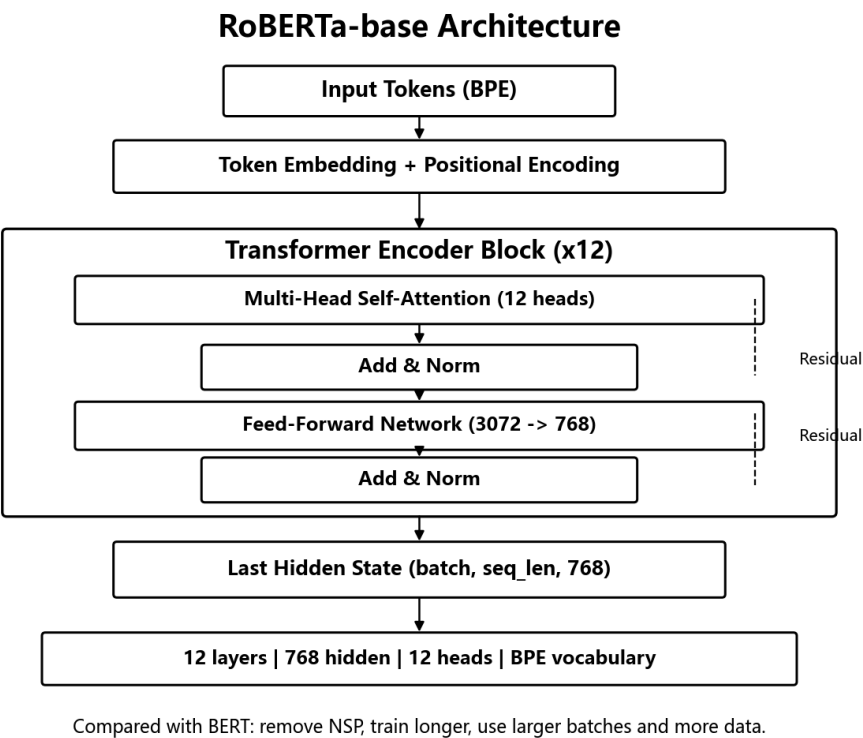


图 3.2 RoBERTa 模型结构示意图

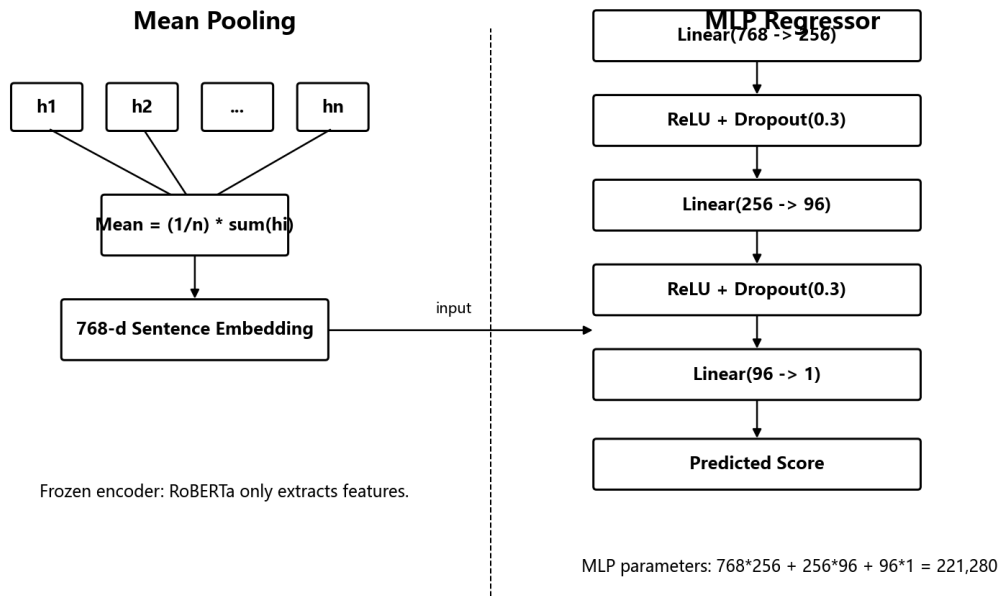


图 3.3 Mean Pooling 与 MLP 回归器结构图

QWK 的计算公式如下：

$$\kappa = 1 - \frac{\sum_i \sum_j w_{ij} O_{ij}}{\sum_i \sum_j w_{ij} E_{ij}}, \quad w_{ij} = \frac{(i - j)^2}{(N - 1)^2}$$

其中，O 表示模型分数与人工分数形成的实际混淆矩阵，E 表示随机一致性假设下的期望矩阵，w 为距离惩罚权重，N 为评分等级数。

3.3 功能模块设计

系统功能模块可划分为前端交互模块、评分接口模块、模型推理模块、文本分析模块、历史记录模块和导出模块。前端交互模块负责用户输入和结果展示；评分接口模块负责接收单篇文本或批量文件请求；模型推理模块负责完成 RoBERTa 编码和 MLP 预测；文本分析模块负责生成字符数、词数、句子数、段落数、平均句长、词汇丰富度和高频词；历史记录模块负责保存与查询评分记录；导出模块负责将评分详情转换为 CSV 文件。

模块划分遵循职责单一原则。模型加载与推理封装在 model_service.py 中，避免后端接口直接操作模型细节；数据库访问封装在 storage.py 中，避免业务接口直接编写重复 SQL；Flask 应用入口 app.py 负责接口注册和业务编排；Streamlit 前端 streamlit_app.py 负责页面布局、用户输入和后端 API 调用。这样的划分使系统便于调试，也便于后续替换模型或迁移前端框架。

表 3.1 系统功能模块设计表

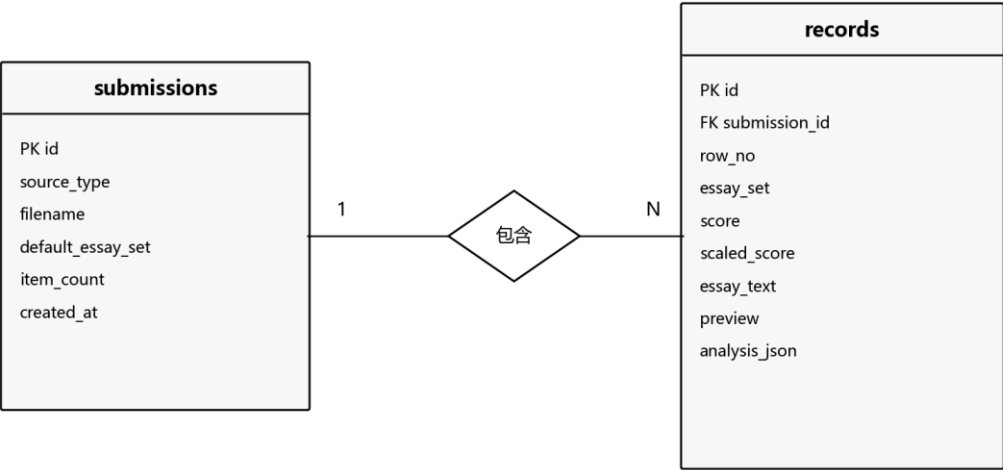
模块名称	对应文件/位置	主要职责
前端交互模块	streamlit_app.py	页面导航、输入控件、结果展示、历史列表、导出按钮
评分接口模块	app.py	提供 /api/score/text 与 /api/score/file 接口
模型推理模块	model_service.py	加载 tokenizer、RoBERTa、MLP，完成分数预测
文本分析模块	app.py:text_analysis	统计文本长度、句子、段落、词汇丰富度和高频词
历史记录模块	storage.py	SQLite 双表增删查和分页筛选
文件归档模块	app.py:ScoreFileResource	保存上传 CSV 到 uploads 目录
导出模块	streamlit_app.py:build_export_csv	构造兼容 Excel 的 UTF-8 BOM CSV

3.4 数据库设计

系统采用 SQLite 作为嵌入式数据库，数据库文件位于 aes/data/history.db。数据库设计围绕评分提交和评分明细两个核心实体展开。submissions 表记录一次提交的总体信息，包括提交类型、上传文件名、默认题目组、作文数量和创建时间；records 表记录每篇作文的评分明细，包括行号、题目组、预测分数、标准化分数、作文原文、预览文本和文本分析 JSON。

该设计将一次提交与多篇作文明细分离，能够同时支持单篇评分和批量评分。单篇评分在 submissions 中对应一条 source_type 为 text 的记录，并在 records 中对应一条明细；批量评分在 submissions 中对应一条 source_type 为 file 的记录，并在 records 中对应多条明细。历史列表查询只需读取 submissions 表，详情查看再读取 records 表，从而减少列表页面的数据传输量。

系统数据库 ER 图如图 3.4 所示。submissions 与 records 之间是一对多关系，一次评分提交可以包含一篇或多篇作文；records 表中的 submission_id 是外键字段，指向 submissions 表的 id。由于系统当前规模较小，未单独拆分用户表、模型版本表和文件表，而是将提交来源、上传文件名和默认题目组直接保存在 submissions 中，以降低数据库复杂度并满足工程实践原型系统的持久化需求。



一次评分提交可包含一条或多条作文明细；records.submission_id 关联 submissions.id。

图 3.4 历史记录数据库 ER 图

表 3.2 submissions 表结构

字段名称	数据类型	说明
id	INTEGER	主键，自增提交编号
source_type	TEXT	提交类型，text 或 file
filename	TEXT	上传归档文件名，单篇评分为空
default_essay_set	INTEGER	默认题目组，unknown 模式为空
item_count	INTEGER	本次提交包含的作文数量
created_at	TEXT	提交创建时间

表 3.3 records 表结构

字段名称	数据类型	说明
id	INTEGER	主键，自增明细编号
submission_id	INTEGER	关联 submissions.id
row_no	INTEGER	批量文件中的行号或单篇记录编号
essay_set	INTEGER	题目组编号，unknown 以 -1 保存
score	INTEGER	最终预测分数
scaled_score	REAL	模型输出的标准化分数
essay_text	TEXT	作文原文
preview	TEXT	作文预览文本

analysis_json	TEXT	文本分析结果 JSON
---------------	------	-------------

3.5 接口设计

表 3.4 后端 RESTful API 设计表

请求方法	接口路径	接口说明
GET	/api/health	健康检查，返回模型路径、数据库路径和可用题目组
POST	/api/score/text	单篇作文评分，接收 essay 和 essay_set
POST	/api/score/file	批量文件评分，接收 CSV 文件和 essay_set
GET	/api/history	历史记录分页查询，支持题目组和提交类型筛选
GET	/api/history/<submission_id>	查询某次提交的评分详情
DELETE	/api/history/<submission_id>	删除某次提交及其评分明细

第 4 章 系统详细设计与实现

4.1 开发环境与目录结构

本系统主要使用 Python 语言开发，核心依赖包括 Flask、Flask-RESTful、Streamlit、PyTorch、Transformers、NumPy、Pandas 和 Requests。后端服务默认运行在 127.0.0.1:5000，前端 Streamlit 应用默认运行在 localhost:8501。系统目录以 aes 为核心，backend 保存后端接口、模型服务和数据库访问代码，frontend 保存前端页面代码，data 保存测试数据和历史数据库，uploads 保存批量评分上传文件归档。

项目代码结构简洁，便于工程实践展示与部署。backend/app.py 是 Flask 应用入口，采用 create_app 工厂函数创建应用并注册 Resource；backend/model_service.py 定义 MLP、EssayScoreResult 和 EssayScoringService，封装模型加载、编码、预测和分数反标准化逻辑；backend/storage.py 定义 HistoryStore，封装 SQLite 初始化、保存、查询、统计和删除操作；frontend/streamlit_app.py 实现三页面交互界面。

表 4.1 项目主要目录与文件说明

文件或目录	作用
aes/backend/app.py	Flask-RESTful API 入口，定义评分、历史和健康检查接口
aes/backend/model_service.py	RoBERTa+MLP 模型加载与推理逻辑
aes/backend/storage.py	SQLite 历史记录存储和 CRUD 操作
aes/frontend/streamlit_app.py	Streamlit 前端，包含单篇评分、批量评分和历史记录页面
aes/data/history.db	历史记录数据库
aes/data/test_batch.csv	批量评分测试样例
aes/uploads/	上传文件归档目录
aes/requirements.txt	系统运行依赖

4.2 系统类结构设计

系统详细设计首先从类结构入手。系统后端核心类包括 EssayScoringService、MLP、EssayScoreResult 和 HistoryStore；前端虽然以 Streamlit 脚本方式实现，但也可以抽象为前端交互模块，负责调用后端 API、构造导出文件和维护页面状态。系统类图如图 4.1 所示。

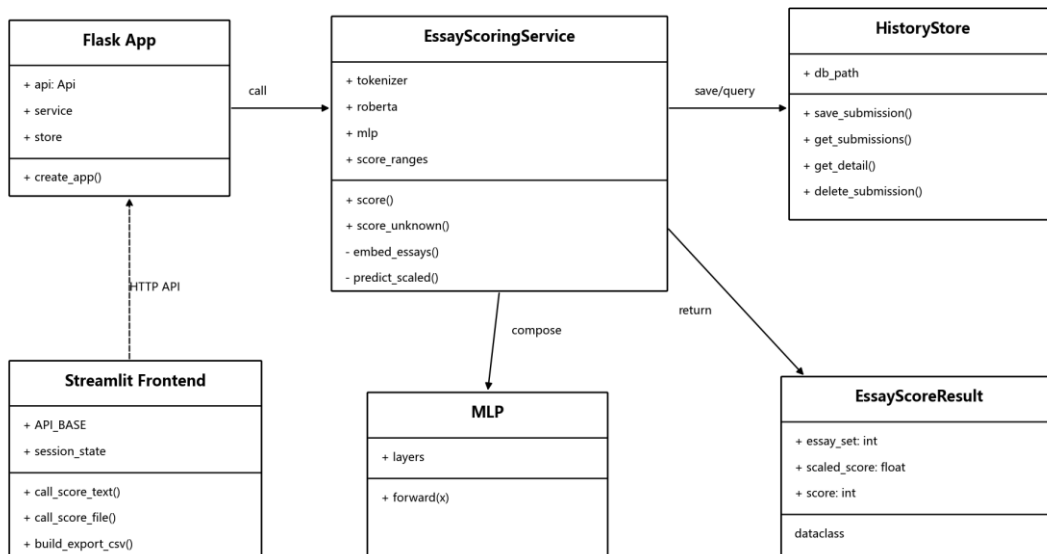


图 4.1 系统核心类图

图 4.1 中，Flask App 通过组合方式持有 EssayScoringService 和 HistoryStore 两个核心对象。EssayScoringService 负责模型推理，与 MLP 存在组合关系，并以 EssayScoreResult 作为已知题目评分结果的数据结构。HistoryStore 负责数据库访问，包括保存提交、查询列表、查询详情和删除记录等方法。Streamlit 前端不直接访问数据库和模型，而是通过 HTTP API 调用 Flask App，从而保证前后端职责分离。

这种类结构设计的优点在于模块边界清晰。模型推理逻辑集中在 EssayScoringService 中，后续替换为其他预训练模型时，只需要保持 score 和 score_unknown 等对外方法稳定；存储逻辑集中在 HistoryStore 中，后续从 SQLite 迁移到 MySQL 或 PostgreSQL 时，可以减少对接口层和前端层的影响；Flask App 只承担业务编排职责，避免接口函数中混入过多模型和数据库细节。

4.3 单篇评分顺序流程设计

单篇评分是系统最基础也是最常用的流程。为了描述对象之间的调用顺序，本文绘制了单篇评分顺序图，如图 4.2 所示。该流程从用户在前端提交作文开始，经过 Streamlit 前端、Flask API、模型服务和 SQLite 数据库，最终将评分结果返回并展示给用户。

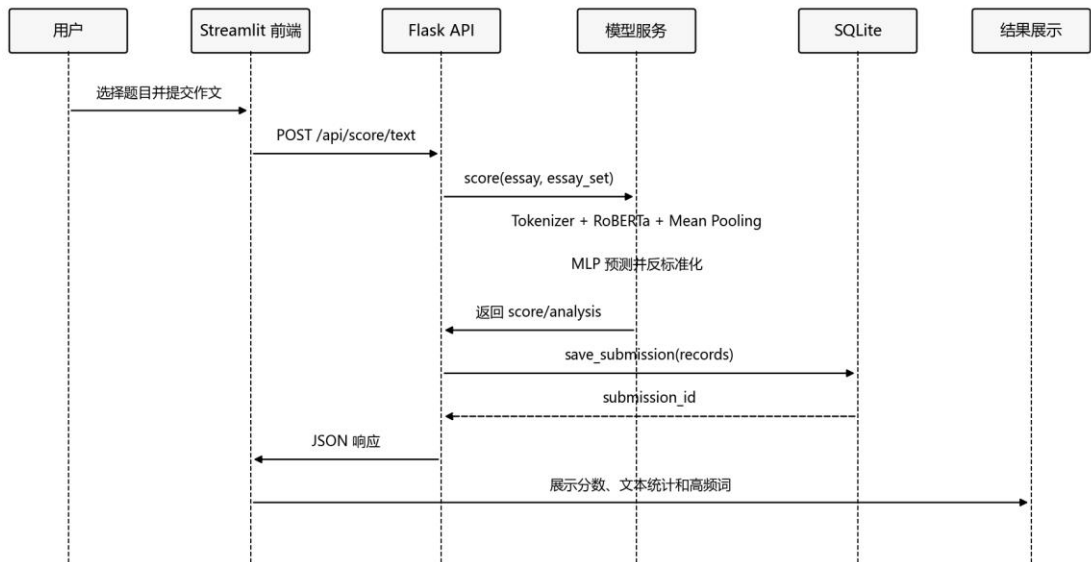


图 4.2 单篇评分顺序图

如图 4.2 所示，用户首先选择作文题目并提交作文文本，Streamlit 前端将数据封装为 JSON 请求并调用 `/api/score/text` 接口。Flask API 在完成参数校验后调用 `EssayScoringService.score` 或 `score_unknown` 方法。模型服务内部依次执行 tokenizer 编码、RoBERTa 语义表示提取、Mean Pooling、MLP 回归预测和分数反标准化。得到评分结果后，Flask API 调用 `HistoryStore.save_submission` 保存提交记录和作文明细，然后将 `submission_id`、`score`、`scaled_score`、`essay_set` 和 `analysis` 返回给前端。前端最终展示预测分数、文本统计表和高频词图表。

该顺序流程体现了系统运行时的关键控制路径。模型推理与历史存储在后端串行完成，保证用户看到的评分结果已经持久化；前端只负责展示和交互，不保存业务状态；数据库只保存结构化评分记录，不参与模型计算。这样的设计有利于定位问题，例如评分失败可优先检查模型服务，历史记录异常可优先检查 `HistoryStore` 和 `SQLite` 文件。

4.4 多篇评分顺序流程设计

多篇评分主要面向教师一次性处理班级作业或测试作文的场景。为了说明批量文件上传、解析、循环评分和结果保存之间的调用关系，本文绘制了多篇评分顺序图，如图 4.3 所示。

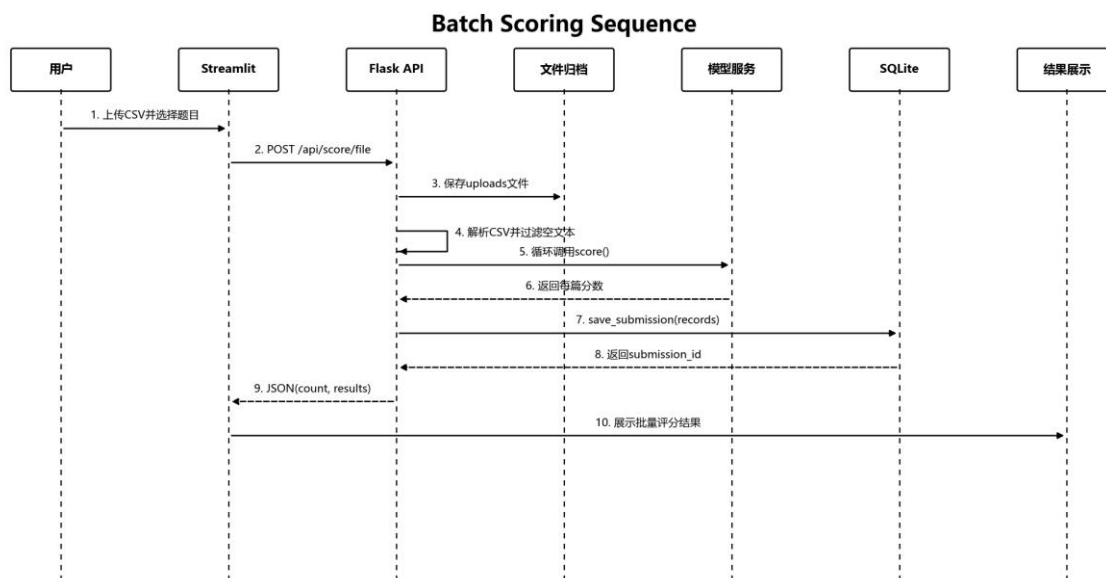


图 4.3 多篇评分顺序图

如图 4.3 所示，用户在 Streamlit 前端上传 CSV 文件并选择题目组，前端通过 POST /api/score/file 将文件和参数提交给 Flask API。后端首先校验文件类型，将上传文件保存到 uploads 目录，随后使用 pandas 读取 CSV 数据，按作文列提取非空文本并生成待评分列表。对于列表中的每篇作文，Flask API 调用 EssayScoringService.score 或 score_unknown 完成模型推理，得到分数、标准化分数和文本分析结果。全部作文处理完成后，后端将同一批次的 records 交给 HistoryStore.save_submission 统一保存，并将 submission_id、count 和 results 返回给前端。前端根据返回结果展示多篇作文的评分表格，并支持后续导出。

多篇评分流程与单篇评分流程的主要区别在于文件归档、CSV 解析和循环评分三个环节。系统将一次上传视为一个 submission，将每篇作文视为该 submission 下的一条 record，这样既能保留批量处理的整体信息，也能在历史详情中查看每篇作文的独立评分结果。

4.5 历史记录顺序流程设计

历史记录功能用于追踪系统已经完成的评分任务，支持分页查询、条件筛选、详情查看、CSV 导出和记录删除。为了说明历史数据在前端、接口层和存储层之间的交互过程，本文绘制了历史记录顺序图，如图 4.4 所示。

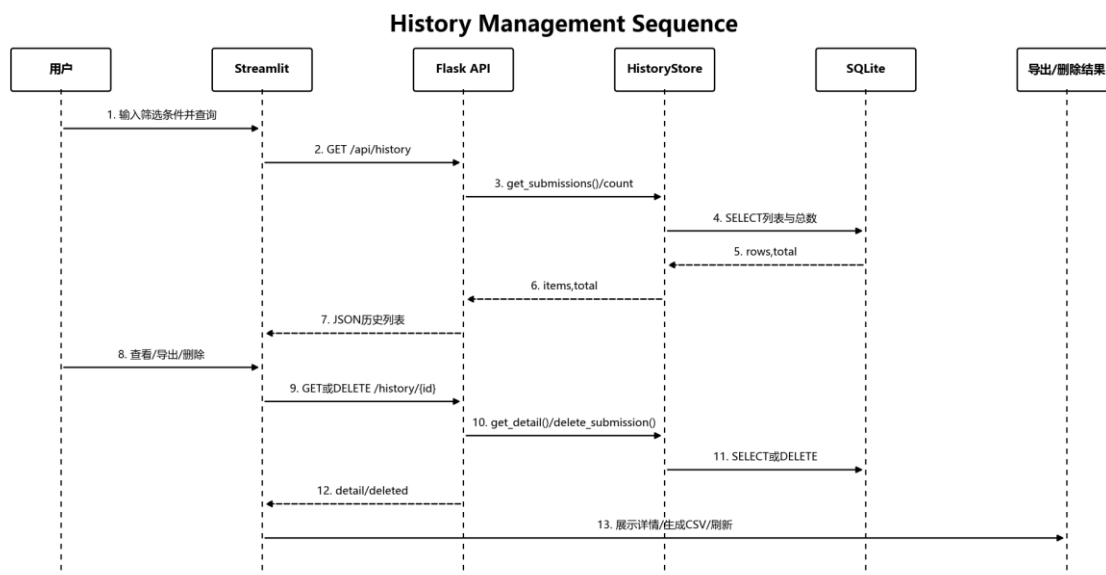


图 4.4 历史记录顺序图

如图 4.4 所示，用户在历史记录页面输入筛选条件后，Streamlit 前端调用 GET /api/history 接口，并传入 limit、offset、essay_set 和 source_type 等参数。Flask API 将查询请求转交给 HistoryStore，HistoryStore 分别执行列表查询和总数统计，并把 items、total、limit、offset 等信息返回给前端。当前端需要查看某条记录详情时，会继续调用 GET /api/history/{id}，后端通过 get_submission_detail 读取提交主表和作文明细表，并返回 records 列表。若用户选择删除记录，前端调用 DELETE /api/history/{id}，后端通过 delete_submission 删除对应提交及其明细。CSV 导出在前端基于详情数据生成，避免后端再维护额外的导出接口。

该流程将历史列表和历史详情拆分为两个阶段：列表查询只返回摘要信息，详情查询才返回完整作文文本和分析结果，能够减少历史页面首次加载时的数据量。删除操作通过后端统一访问 HistoryStore 完成，保证提交记录和作文明细的一致性。

4.6 模型服务详细实现

模型服务由 EssayScoringService 类实现。服务初始化时读取模型包文件，优先从模型包中加载 best_model、tokenizer、roberta_config、roberta_state_dict、score_ranges 和 score_scalers 等信息。若模型包缺少分数范围信息，服务会尝试从训练数据 training_set_rel3.tsv 中统计各题目的最小分和最大分。该设计提高了模型包兼容性，使后端既能加载完整训练产物，也能在轻量 checkpoint 下补全必要元数据。

RoBERTa 构建逻辑分为两种情况：如果模型包中保存了 `roberta_config` 和 `roberta_state_dict`，服务直接恢复训练时使用的 RoBERTa 参数；否则使用 `RobertaModel.from_pretrained("roberta-base")` 作为回退方案。Tokenizer 也优先使用模型包内序列化对象，并通过一次健康检查调用验证其可用性；若序列化对象与当前 Transformers 版本不兼容，则回退到 `RobertaTokenizerFast.from_pretrained("roberta-base")`。

推理流程中，`_embed_essays` 方法按 batch 对作文文本进行编码，最大长度设置为 512，并使用 padding 和 truncation 处理变长文本。RoBERTa 输出的 `last_hidden_state` 在 token 维度取均值后转换为 NumPy 数组。`_predict_scaled` 方法再将向量输入 MLP，得到标准化分数。对于已知题目组，`score` 方法调用 `_inverse_scale_score` 反标准化并裁剪到合法分数范围；对于 unknown 题目，`score_unknown` 方法遍历所有题目组的 `scaler` 和分数范围，将各组反标准化结果归一化到 0 - 100 后取均值，得到综合评分。

4.7 后端服务详细实现

后端服务使用 Flask-RESTful 的 Resource 组织接口。HealthResource 用于检查服务状态，返回模型路径、数据库路径和可用题目组。ScoreTextResource 处理单篇评分请求，从 JSON 中读取 `essay` 和 `essay_set`，完成空文本校验、题目组解析、模型评分、文本分析和历史记录保存。评分结果包括 `submission_id`、`score`、`scaled_score`、`essay_set` 和 `analysis`，前端据此展示分数和文本统计。

ScoreFileResource 处理批量评分请求。接口要求 form-data 中包含 `file` 字段，同时读取 `essay_set` 参数。后端会将上传文件保存到 uploads 目录，文件名包含时间戳，便于追溯原始输入。当前实现支持 CSV 文件，解析时先尝试 pandas 的自动分隔符识别，失败后回退到逗号分隔读取。系统将 CSV 中所有非空单元格视为作文文本，逐篇调用模型服务评分，并将每篇作文的结果组装为 `results` 数组。

后端还实现了历史记录相关接口。HistoryListResource 支持 `limit`、`offset`、`essay_set` 和 `source_type` 参数，能够实现分页、题目组筛选和提交类型筛选。HistoryDetailResource 支持 GET 和 DELETE 两种方法，分别用于查看某次提交详情和删除记录。删除操作先判断 `submissions` 中记录是否存在，再删除 `records` 中的明细，最后删除 `submissions` 主记录。

4.8 前端页面详细实现

前端使用 Streamlit 实现，整体采用侧边栏导航，包含“单篇评分”“批量评分”和“历史记录”三个页面。页面通过 `st.sidebar.radio` 切换模式，通过 `st.session_state` 保

存交互状态。系统在页面顶部设置 `page_title`、`page_icon` 和 `layout`，并通过自定义 CSS 隐藏部分默认工具栏、调整侧边栏选中样式、统一按钮颜色和表格对齐方式，使界面更加简洁。

单篇评分页面左侧提供题目选择框和作文文本输入框，右侧展示评分结果。用户点击“开始评分”后，前端调用 `/api/score/text` 接口。若作文为空，页面提示用户输入内容；若评分成功，页面展示提交编号、预测分数、文本统计表和高频词柱状图。分数展示会根据题目组最大分拼接为“预测分/满分”的形式，例如 Set 1 显示为 9/12；unknown 模式显示为 0 - 100 范围分数。

批量评分页面提供 CSV 上传控件和题目选择框。用户上传文件后，前端会先调用本地 `extract_csv_essays` 函数预览 CSV 中解析出的作文内容，方便用户确认文件格式。点击“开始评分”后，前端调用 `/api/score/file` 接口，并按返回 `results` 逐篇展示编号、分数、文本统计和高频词。该页面适合教师一次性处理多篇学生作业。

历史记录页面支持编号查询、题目组筛选、提交类型筛选、分页、查看、导出和删除。列表页面使用多列布局模拟表格，展示序号、编号、作文类型、作文题目、数量、创建时间和操作按钮。点击查看后进入详情页面，展示对应作文原文和评分分析；点击导出时调用 `build_export_csv` 生成 CSV；点击删除时弹出确认对话框，避免误删。

4.9 历史记录与导出实现

历史记录由 `HistoryStore` 类统一管理。初始化时，系统调用 `_init_db` 创建 `submissions` 和 `records` 两张表，若表已存在则不会重复创建。保存记录时，`save_submission` 先向 `submissions` 插入一次提交的总体信息，获取自增 `submission_id` 后，再使用 `executemany` 将所有作文明细批量写入 `records`。所有数据库操作均通过 `sqlite3` 连接上下文完成，保证提交和资源释放。

查询列表时，`get_submissions` 根据 `default_essay_set` 和 `source_type` 动态拼接 `where` 条件，并使用 `LIMIT/OFFSET` 实现分页。`count_submissions` 用于统计筛选条件下的总数，从而支持前端计算总页数。查询详情时，`get_submission_detail` 先查询提交主记录，再按 `row_no` 升序查询所有明细，并将 `analysis_json` 反序列化为字典后返回给前端。

导出功能在前端实现。`build_export_csv` 首先调用详情接口获取 `records`，然后为每条记录构造“编号、预测成绩、文本分析、高频词、文章”五列。为了避免 Excel 将编号 1-2 或分数 10/12 误识别为日期，系统将编号和预测成绩写成 Excel 文本公式形式。最终 CSV 使用 `utf-8-sig` 编码，能够兼容中文办公场景。

第 5 章 系统测试

5.1 测试环境与方法

系统测试从模型评估、接口测试、功能测试和非功能测试四个方面展开。模型评估侧重验证 RoBERTa+MLP 在 ASAP 数据集上的评分一致性；接口测试侧重验证后端 RESTful API 的请求和响应是否符合设计；功能测试侧重验证前端页面是否能够完成单篇评分、批量评分、历史查询、导出和删除等业务流程；非功能测试侧重验证系统的易用性、可靠性、可维护性和响应时间是否满足工程实践要求。

测试数据主要包括 ASAP 训练数据、系统提供的 test_batch.csv 批量测试样例以及历史数据库中的实际评分记录。项目历史数据库中已保存 36 次提交和 39 条评分明细，说明系统在开发测试过程中已经完成多轮单篇和批量评分验证。

表 5.1 测试环境表

测试项	配置说明
操作系统	Windows / 本地开发环境
后端框架	Flask 3.0.3, Flask-RESTful 0.3.10
前端框架	Streamlit 1.38.0
模型框架	PyTorch 2.2+, Transformers 4.40+
数据处理	NumPy 1.26+, Pandas 2.2+
数据库	SQLite
评估指标	Quadratic Weighted Kappa, MSELoss

5.2 模型评估测试

模型评估采用 5-fold 交叉验证。每一折中，训练集用于拟合各题目组的分数标准化参数和训练 MLP 回归器，验证集用于计算预测分数与人工分数之间的 QWK。QWK 能够衡量两个评分者之间的一致性，并对预测分与人工分之间的距离进行二次惩罚，因此适合作文评分这类有序分数任务。

根据训练 notebook 和结题汇报，RoBERTa+MLP 模型的 5 折平均 QWK 约为 0.7531，最好折达到 0.7684；结题汇报中展示的整体结果为 QWK=0.7585。人工评分员之间的一致性约在 0.70 - 0.80 区间，模型结果已达到可比较水平。各折结果存在一定波动，主要受题目组差异、长文本截断和评分区间分布影响。

表 5.2 模型评估结果表

评估项目	结果或配置
评价指标	Quadratic Weighted Kappa
交叉验证	5-fold, random_state=2022
训练轮数	20 epochs

优化器	Adam, lr=3e-4
平均 QWK	约 0.7531
最好折 QWK	约 0.7684
汇报结果	QWK=0.7585 左右
人工一致性参考	约 0.70 - 0.80

ASAP 数据集 - 类别分布

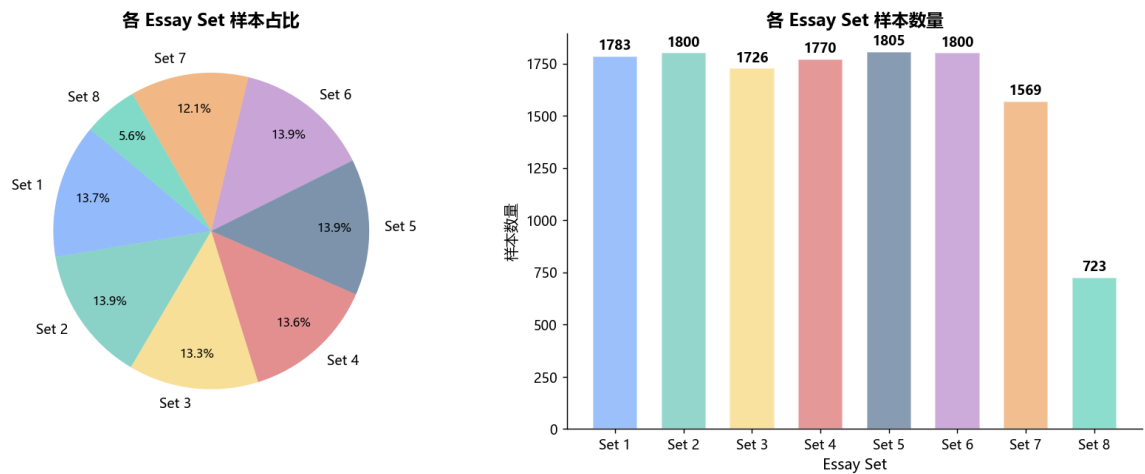


图 5.1 ASAP 各题目组样本分布

ASAP 数据集 - 各 Essay Set 分数分布

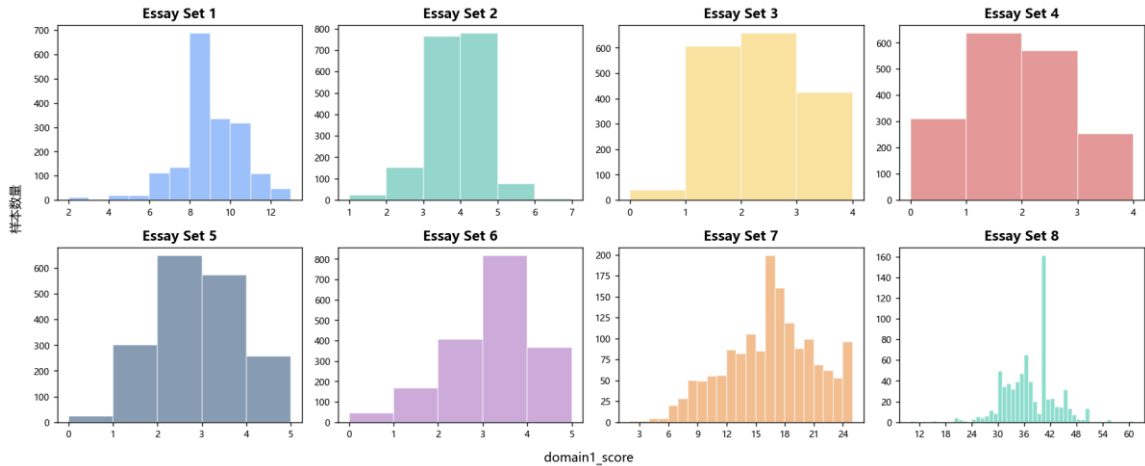


图 5.2 ASAP 各题目组分数分布

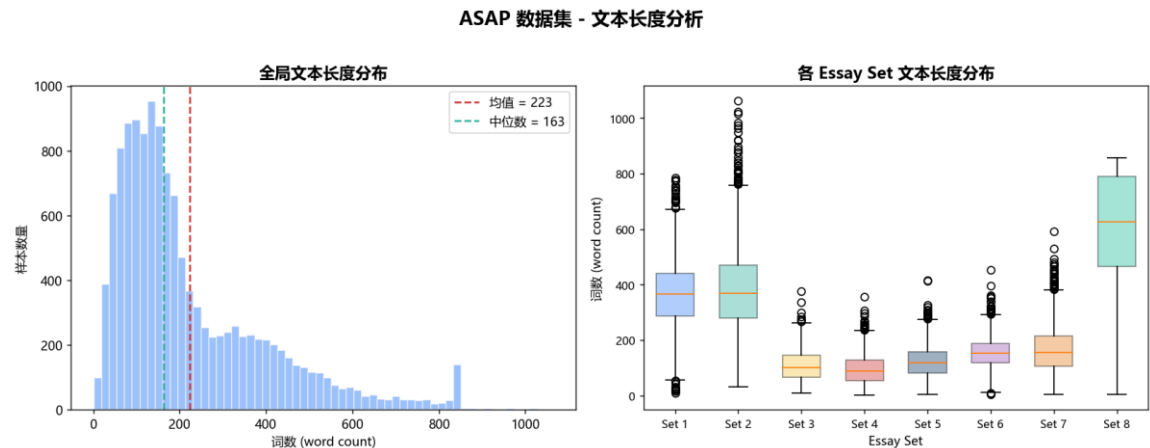


图 5.3 ASAP 文本长度分布

5.3 功能测试

表 5.3 系统功能测试用例表

测试项	测试步骤	预期结果	测试结论
健康检查	访问 /api/health	返回 status=ok、模型路径、数据库路径和题目组	通过
单篇评分	输入非空作文并选择 Set 1	返回 submission_id、score、scaled_score 和 analysis	通过
空文本校验	单篇评分提交空字符串	返回 essay required 错误或前端提示输入内容	通过
批量评分	上传 test_batch.csv 并选择题目组	逐篇返回评分结果并保存上传文件归档	通过
非法文件类型	上传非 CSV 文件	返回 Only .csv is supported	通过
历史列表	访问 /api/history 并设置分页参数	返回 items、total、limit 和 offset	通过
历史详情	查询存在的 submission_id	返回提交主信息和 records 明细	通过
历史筛选	按 essay_set 或 source_type 筛选	仅返回符合条件的记录	通过
CSV 导出	点击历史记录导出按钮	生成包含编号、分数、文本分析、高频词和文章的 CSV	通过
删除记录	确认删除指定	删除提交记录及其明	通过

	submission_id	细，再次查询返回不存在	
--	---------------	-------------	--

功能测试结果表明，系统核心业务流程能够闭环运行。单篇评分和批量评分均能调用模型服务并返回文本分析结果；历史记录能够正确保存提交与明细；分页、筛选、查看、导出和删除功能满足设计要求。系统对空文本、缺少上传文件和不支持文件类型等异常场景也提供了错误提示。

5.4 非功能测试

非功能测试主要从评分一致性、响应时间、可靠性、易用性和可维护性五个方面进行。评分一致性已通过 QWK 指标验证。响应时间方面，单篇评分的耗时主要来自 RoBERTa 前向计算，批量评分耗时随作文数量线性增长；对于课程作业场景，系统更适合中小规模批量评分。可靠性方面，后端对必填参数和文件类型进行校验，并将模型调用异常统一转换为错误响应。易用性方面，前端将评分、批量处理和历史管理拆分为三个页面，降低操作复杂度。可维护性方面，系统代码将模型、接口、存储和前端分离，便于后续迭代。

需要说明的是，当前系统作为工程实践原型，尚未进行大规模并发压力测试。后续若部署到真实教学平台，应补充并发请求、批量大文件、长文本极端输入和模型服务内存占用等测试，并根据测试结果考虑引入 GPU 推理、异步任务队列、缓存或模型压缩等优化措施。

表 5.4 非功能测试结果表

测试维度	测试说明	结论
评分一致性	5 折交叉验证 QWK 约 0.75	达到预期
响应时间	单篇评分依赖模型前向计算，小规模使用可接受	基本满足
可靠性	空文本、缺文件、非 CSV 文件均有校验	满足
易用性	三页面导航、结果表格和图表展示清晰	满足
可维护性	模型服务、API、存储和前端分离	满足
可扩展性	RESTful API 可对接第三方系统	满足
兼容性	导出 CSV 使用 UTF-8 BOM	满足

第 6 章 总结与展望

6.1 总结

本文针对文本作业人工评分效率低、评分一致性不足和基础反馈生成滞后的问题，设计并实现了一个基于深度学习的文本作业自动评分系统。系统以 RoBERTa 预训练语言模型为语义编码器，通过 Mean Pooling 获取固定长度作文向量，并使用 MLP 回归器预测分数。训练过程中采用按题目组独立标准化、5 折交叉验证和 QWK 指标评估模型一致性，实验结果显示模型能够取得约 0.75 的平均 QWK，达到与人工评分员相近的水平。

在系统实现方面，本文完成了 Flask-RESTful 后端、PyTorch/Transformers 模型推理服务、SQLite 历史记录存储和 Streamlit 前端界面的集成。系统支持单篇评分、批量 CSV 评分、文本统计分析、历史记录查询筛选、详情查看、CSV 导出和删除等功能，形成了从文本输入、模型评分、结果展示到数据持久化的完整闭环。

本文的工程实践意义在于将 AES 模型从 notebook 实验推进到可交互系统原型。模型层面验证了预训练语言模型在作文评分任务上的有效性；系统层面实现了清晰的模块划分和轻量化部署方案；应用层面为教师处理文本作业提供了辅助工具雏形。

表 6.1 本文主要成果总结表

维度	主要成果
模型设计	RoBERTa frozen encoder + Mean Pooling + MLP 回归器
训练策略	分题目组标准化、MSELoss、Adam、5-fold 交叉验证
系统架构	Streamlit + Flask-RESTful + PyTorch/Transformers + SQLite
核心功能	单篇评分、批量评分、文本分析、历史记录、导出和删除
评估结果	平均 QWK 约 0.75，接近人工评分一致性水平
工程价值	形成可运行、可扩展、可追溯的自动评分系统原型

6.2 展望

5. 增强可解释性反馈。当前系统提供文本统计和高频词等基础反馈，后续可引入大语言模型或可解释性方法，生成关于内容完整性、结构组织、语言表达和逻辑连贯性的自然语言反馈。
6. 支持多维评分。现阶段系统输出单一总分，后续可扩展为内容、组织、语言、语法、词汇等多个维度评分，更贴近教师评分量表。

7. 提升长文本处理能力。RoBERTa 最大输入长度为 512 token，长作文存在截断问题。后续可尝试 Longformer、分段编码、层次化 pooling 或滑动窗口策略。
8. 扩展中文和多语言作业评分。当前模型主要面向英文作文，后续可引入中文预训练模型和本地课程数据，支持语文、历史、政治等中文论述题评分。
9. 完善模型版本管理。后续可增加模型版本表、A/B 测试、评分日志和回滚机制，使系统能够在真实教学场景中持续迭代。
10. 提升部署与性能。若面向多人同时使用，可将模型服务独立部署，使用 GPU 推理、异步任务队列、请求缓存和服务监控，提高系统吞吐量和稳定性。

参考文献

- [1] Misgna H, On B W, Lee I, Choi G S. A survey on deep learning-based automated essay scoring and feedback generation[J]. Artificial Intelligence Review, 2025, 58:36.
- [2] Yang R, Cao J, Wen Z, Wu Y, He X. Enhancing Automated Essay Scoring Performance via Fine-tuning Pre-trained Language Models with Combination of Regression and Ranking[C]. Findings of EMNLP, 2020:1560–1569.
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]. Advances in Neural Information Processing Systems, 2017.
- [4] Devlin J, Chang M W, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]. NAACL-HLT, 2019.
- [5] Liu Y, Ott M, Goyal N, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach[EB/OL]. arXiv:1907.11692, 2019.
- [6] Taghipour K, Ng H T. A Neural Approach to Automated Essay Scoring[C]. EMNLP, 2016.
- [7] Dong F, Zhang Y, Yang J. Attention-based Recurrent Convolutional Neural Network for Automatic Essay Scoring[C]. CoNLL, 2017.
- [8] Hamner B, Morgan J, Lynn H, Shermis M. The Hewlett Foundation: Automated Essay Scoring Dataset[EB/OL]. Kaggle, 2012.
- [9] Cohen J. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit[J]. Psychological Bulletin, 1968, 70(4):213–220.
- [10] Flask Documentation. Flask Web Development Framework[EB/OL].
- [11] Hugging Face. Transformers Documentation[EB/OL].
- [12] PyTorch Documentation. PyTorch Machine Learning Framework[EB/OL].
- [13] Streamlit Documentation. Streamlit App Framework[EB/OL].